

Pandas Complete Function Reference

Every function from Series creation to groupby & cut —
with clear one-line explanations and real outputs.

beginner → intermediate | zero functions skipped

pandas	Python	Level	Coverage
2.x	3.x	Beginner → Intermediate	100%

Table of Contents

1. Creating a DataFrame & Series
2. Viewing & Inspecting Data
3. Selecting & Accessing Data
4. Filtering & Boolean Indexing
5. Adding & Removing Columns / Rows
6. Renaming & Reindexing
7. Sorting
8. Handling Missing Values
9. Data Types & Casting
10. String Methods (.str accessor)
11. DateTime Methods (.dt accessor)
12. Aggregation & Statistics
13. Applying Functions (apply / map)
14. Merging & Joining
15. Reshaping (pivot / melt / stack)
16. Binning with pd.cut()
17. GroupBy

1 · Creating a DataFrame & Series

■ `pd.DataFrame()`

Creates a 2-D table (rows × columns) from a dict, list-of-lists, NumPy array, or another DataFrame.

CODE

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Score': [88.5, 92.0, 79.3]
})

print(df)
```

OUTPUT

```
   Name  Age  Score
0  Alice   25   88.5
1   Bob   30   92.0
2 Charlie   35   79.3
```

■ `pd.Series()`

Creates a 1-D labelled array — the building block of every DataFrame column.

CODE

```
s = pd.Series([10, 20, 30, 40], name='values')

print(s)
```

OUTPUT

```
0    10
1    20
2    30
3    40

Name: values, dtype: int64
```

■ `pd.read_csv()`

Reads a CSV file from disk (or a URL) and returns a DataFrame.

CODE

```
df = pd.read_csv('data.csv') # basic

df = pd.read_csv('data.csv',

sep=';', # custom delimiter

index_col='ID', # set index column

parse_dates=['Date'])# auto-parse dates

print(df.head(2))
```

OUTPUT

```
Name Age Score

0 Alice 25 88.5

1 Bob 30 92.0
```

■ `pd.read_excel()`

Reads an Excel file (.xlsx / .xls) into a DataFrame.

CODE

```
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')

print(df.head(2))
```

OUTPUT

```
Name Age Score

0 Alice 25 88.5

1 Bob 30 92.0
```

■ `pd.DataFrame.to_csv()`

Saves a DataFrame to a CSV file on disk.

CODE

```
df.to_csv('output.csv', index=False)

# index=False prevents writing the row numbers
```

OUTPUT

```
# File 'output.csv' created successfully
```

■ `pd.DataFrame.to_excel()`

Saves a DataFrame to an Excel file.

CODE

```
df.to_excel('output.xlsx', index=False, sheet_name='Results')
```

OUTPUT

```
# File 'output.xlsx' created successfully
```

■ `pd.DataFrame()` from list-of-dicts

Each dict becomes one row — a very readable way to hand-build small tables.

CODE

```
rows = [  
    {'City': 'Delhi', 'Pop_M': 32},  
    {'City': 'Mumbai', 'Pop_M': 21},  
]  
  
df2 = pd.DataFrame(rows)  
  
print(df2)
```

OUTPUT

```
City Pop_M  
0 Delhi 32  
1 Mumbai 21
```

■ `pd.DataFrame.copy()`

Makes an independent copy so edits on the copy don't affect the original.

CODE

```
df_copy = df.copy()  
  
df_copy['Age'] = 0 # does NOT change df  
  
print(df['Age'].values)
```

OUTPUT

```
[25 30 35]
```

■ Always copy before mutating, to avoid the *SettingWithCopyWarning*.

2 · Viewing & Inspecting Data

■ df.head(n)

Shows the first n rows (default 5) — quick sanity check after loading.

CODE

```
print(df.head(2))
```

OUTPUT

```
Name Age Score
0 Alice 25 88.5
1 Bob 30 92.0
```

■ df.tail(n)

Shows the last n rows — useful to verify the end of a large file.

CODE

```
print(df.tail(2))
```

OUTPUT

```
Name Age Score
1 Bob 30 92.0
2 Charlie 35 79.3
```

■ df.shape

Returns a (rows, columns) tuple — tells you the size of the DataFrame at a glance.

CODE

```
print(df.shape)
```

OUTPUT

```
(3, 3)
```

■ df.dtypes

Shows the data type of every column — great for spotting columns loaded as the wrong type.

CODE

```
print(df.dtypes)
```

OUTPUT

```
Name object
Age int64
Score float64
dtype: object
```

■ df.info()

Prints a compact summary: row count, column names, non-null counts, and dtypes.

CODE

```
df.info()
```

OUTPUT

```
<class 'pandas.core.frame.DataFrame'>

RangeIndex: 3 entries, 0 to 2

Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Name    3 non-null object
1   Age     3 non-null int64
2   Score   3 non-null float64

dtypes: float64(1), int64(1), object(1)
```

■ df.describe()

Computes count, mean, std, min, 25/50/75th percentile, and max for every numeric column.

CODE

```
print(df.describe())
```

OUTPUT

```
Age  Score

count  3.00  3.000000
mean   30.00  86.600000
std     5.00   6.380869
min    25.00  79.300000
25%    27.50  83.900000
50%    30.00  88.500000
75%    32.50  90.250000
max    35.00  92.000000
```

■ df.columns

Returns an Index object listing every column name.

CODE

```
print(df.columns)
```

OUTPUT

```
Index(['Name', 'Age', 'Score'], dtype='object')
```

■ df.index

Returns the row-label index of the DataFrame.

CODE

```
print(df.index)
```

OUTPUT

```
RangeIndex(start=0, stop=3, step=1)
```

■ df.values

Returns the raw NumPy 2-D array underlying the DataFrame (no labels).

CODE

```
print(df.values)
```

OUTPUT

```
[['Alice' 25 88.5]
 ['Bob' 30 92.0]
 ['Charlie' 35 79.3]]
```

■ df.size

Returns the total number of elements (rows × columns).

CODE

```
print(df.size)
```

OUTPUT

```
9
```

■ df.ndim

Returns the number of dimensions — always 2 for a DataFrame, 1 for a Series.

CODE

```
print(df.ndim)
```

OUTPUT

```
2
```

■ `df.memory_usage()`

Shows memory used (in bytes) by each column — handy for optimising large datasets.

CODE

```
print(df.memory_usage(deep=True))
```

OUTPUT

```
Index 128
Name 189
Age 24
Score 24
dtype: int64
```

3 · Selecting & Accessing Data

■ `df['column']`

Selects a single column and returns it as a Series.

CODE

```
print(df['Name'])
```

OUTPUT

```
0 Alice
1 Bob
2 Charlie
Name: Name, dtype: object
```

■ `df[['col1','col2']]`

Selects multiple columns by passing a list — returns a DataFrame.

CODE

```
print(df[['Name', 'Score']])
```

OUTPUT

```
Name Score
0 Alice 88.5
1 Bob 92.0
2 Charlie 79.3
```

■ df.loc[row_label, col_label]

Label-based selection — use actual row/column names (not positions).

CODE

```
print(df.loc[1, 'Name']) # single value

print(df.loc[0:1, 'Name':'Age']) # slice
```

OUTPUT

```
Bob

Name Age
0 Alice 25
1 Bob 30
```

■ df.iloc[row_pos, col_pos]

Position-based selection — use integer positions (like NumPy indexing).

CODE

```
print(df.iloc[0, 1]) # row 0, col 1

print(df.iloc[0:2, 0:2]) # first 2 rows & cols
```

OUTPUT

```
25

Name Age
0 Alice 25
1 Bob 30
```

■ df.at[row, col]

Fastest way to get or set a single scalar value by label.

CODE

```
val = df.at[0, 'Score']

print(val)
```

OUTPUT

```
88.5
```

■ **df.iat[row_pos, col_pos]**

Fastest way to get or set a single scalar value by integer position.

CODE

```
val = df.iat[2, 2]

print(val)
```

OUTPUT

```
79.3
```

■ **df.get('column', default)**

Returns the column (like df['col']) but returns a default instead of raising KeyError.

CODE

```
print(df.get('Salary', 'Not Found'))
```

OUTPUT

```
Not Found
```

4 · Filtering & Boolean Indexing

■ **Boolean mask df[condition]**

Filters rows where a condition is True — the most common pandas pattern.

CODE

```
mask = df['Age'] > 25

print(df[mask])
```

OUTPUT

```
Name Age Score
1 Bob 30 92.0
2 Charlie 35 79.3
```

■ Multiple conditions & and |

Combine conditions with & (AND) or | (OR) — always wrap each condition in ().

CODE

```
result = df[(df['Age'] > 25) & (df['Score'] > 80)]

print(result)
```

OUTPUT

```
   Name  Age  Score
1  Bob   30   92.0
2 Charlie 35   79.3
```

■ df.query()

Filter rows using a readable SQL-like string expression.

CODE

```
print(df.query('Age > 25 and Score > 80'))
```

OUTPUT

```
   Name  Age  Score
1  Bob   30   92.0
2 Charlie 35   79.3
```

■ Series.isin(list)

Checks whether each value is contained in a given list — returns a boolean mask.

CODE

```
print(df[df['Name'].isin(['Alice', 'Charlie'])])
```

OUTPUT

```
   Name  Age  Score
0 Alice  25   88.5
2 Charlie 35   79.3
```

■ Series.between(low, high)

Returns True for values within an inclusive range — shortcut for >= and <=.

CODE

```
print(df[df['Score'].between(85, 93)])
```

OUTPUT

```
   Name  Age  Score
0 Alice  25   88.5
1  Bob   30   92.0
```

■ ~ (NOT / invert mask)

Inverts a boolean mask — selects rows where the condition is False.

CODE

```
print(df[~df['Name'].isin(['Bob'])])
```

OUTPUT

```
Name Age Score
0 Alice 25 88.5
2 Charlie 35 79.3
```

5 · Adding & Removing Columns / Rows

■ df['new_col'] = value

Adds a new column (or replaces an existing one) in-place.

CODE

```
df['Grade'] = ['B', 'A', 'C']

print(df)
```

OUTPUT

```
Name Age Score Grade
0 Alice 25 88.5 B
1 Bob 30 92.0 A
2 Charlie 35 79.3 C
```

■ df.assign()

Returns a new DataFrame with extra columns — safe, no in-place mutation.

CODE

```
df2 = df.assign(Tax=df['Score'] * 0.1)

print(df2[['Name', 'Score', 'Tax']])
```

OUTPUT

```
Name Score Tax
0 Alice 88.5 8.85
1 Bob 92.0 9.20
2 Charlie 79.3 7.93
```

■ df.insert(pos, col, values)

Inserts a column at a specific position index.

CODE

```
df.insert(1, 'Country', ['IN','US','UK'])

print(df.head(2))
```

OUTPUT

```
   Name Country Age Score
0  Alice    IN  25  88.5
1   Bob    US  30  92.0
```

■ df.drop(labels, axis)

Drops specified rows (axis=0) or columns (axis=1) by label.

CODE

```
df_d = df.drop('Grade', axis=1) # drop column

df_d = df_d.drop([2], axis=0) # drop row index 2

print(df_d)
```

OUTPUT

```
   Name Age Score
0  Alice  25  88.5
1   Bob  30  92.0
```

■ df.pop(col)

Removes and returns a column — modifies the DataFrame in-place.

CODE

```
grade_col = df.pop('Grade')

print(grade_col.values)
```

OUTPUT

```
['B' 'A' 'C']
```

■ `pd.concat([df1, df2])`

Stacks DataFrames vertically (axis=0) or side-by-side (axis=1).

CODE

```
df_a = pd.DataFrame({'X': [1, 2]})

df_b = pd.DataFrame({'X': [3, 4]})

print(pd.concat([df_a, df_b], ignore_index=True))
```

OUTPUT

```
X
0  1
1  2
2  3
3  4
```

■ `df.append()` → deprecated; use `pd.concat()`

Old way to add rows — replaced by `pd.concat()` in pandas 2.0.

CODE

```
# Instead of df.append(new_row, ignore_index=True)

new_row = pd.DataFrame([{'Name': 'Dave', 'Age': 28, 'Score': 91}])

df = pd.concat([df, new_row], ignore_index=True)
```

OUTPUT

```
Name Age Score
0 Alice 25 88.5
1 Bob 30 92.0
2 Charlie 35 79.3
3 Dave 28 91.0
```

6 · Renaming & Reindexing

■ df.rename(columns={})

Renames columns (or index labels) using a mapping dict.

CODE

```
df_r = df.rename(columns={'Name': 'Student', 'Score': 'Marks'})

print(df_r.columns.tolist())
```

OUTPUT

```
['Student', 'Age', 'Marks']
```

■ df.columns = [list]

Replaces all column names at once by assigning a new list.

CODE

```
df.columns = ['student', 'age', 'score']

print(df.columns.tolist())
```

OUTPUT

```
['student', 'age', 'score']
```

■ df.set_index(col)

Promotes a column to become the row index.

CODE

```
df_i = df.set_index('Name')

print(df_i)
```

OUTPUT

```
Age Score
Name
Alice 25 88.5
Bob 30 92.0
Charlie 35 79.3
```

■ df.reset_index()

Moves the current index back into a regular column and resets to 0,1,2...

CODE

```
df_back = df_i.reset_index()

print(df_back.head(2))
```

OUTPUT

```
Name Age Score
0  Alice 25  88.5
1  Bob  30  92.0
```

■ df.reindex([new_order])

Reorders rows (or columns) to match a new label list — fills missing with NaN.

CODE

```
df_re = df.reindex([2, 0, 1])

print(df_re)
```

OUTPUT

```
Name Age Score
2  Charlie 35  79.3
0  Alice  25  88.5
1  Bob   30  92.0
```

7 · Sorting

■ df.sort_values(by, ascending)

Sorts rows by one or more column values.

CODE

```
print(df.sort_values('Score', ascending=False))
```

OUTPUT

```
Name Age Score
1  Bob  30  92.0
0  Alice 25  88.5
2  Charlie 35  79.3
```

■ `df.sort_values()` — multi-column

Sorts by multiple columns; ties in the first are broken by the second.

CODE

```
df2 = df.copy()

df2.loc[3] = ['Dave', 30, 95]

print(df2.sort_values(['Age', 'Score'], ascending=[True, False]))
```

OUTPUT

```
Name Age Score
0 Alice 25 88.5
3 Dave 30 95.0
1 Bob 30 92.0
2 Charlie 35 79.3
```

■ `df.sort_index()`

Sorts rows by their index label (useful after concatenating or filtering).

CODE

```
df_shuffled = df.iloc[[2,0,1]]

print(df_shuffled.sort_index())
```

OUTPUT

```
Name Age Score
0 Alice 25 88.5
1 Bob 30 92.0
2 Charlie 35 79.3
```

■ `Series.rank()`

Assigns a rank to each value (handles ties with `method='average'` by default).

CODE

```
print(df['Score'].rank(ascending=False))
```

OUTPUT

```
0 2.0
1 1.0
2 3.0

Name: Score, dtype: float64
```

■ **Series.nlargest(n) / Series.nsmallest(n)**

Returns the n largest (or smallest) values — faster than sort for big Series.

CODE

```
print(df['Score'].nlargest(2))
```

OUTPUT

```
1 92.0
0 88.5

Name: Score, dtype: float64
```

8 · Handling Missing Values

■ **df.isnull() / df.isna()**

Returns a boolean DataFrame of the same shape — True where a value is NaN.

CODE

```
import numpy as np

df_na = df.copy()

df_na.loc[1, 'Score'] = np.nan

print(df_na.isnull())
```

OUTPUT

```
Name Age Score
0 False False False
1 False False True
2 False False False
```

■ **df.notnull() / df.notna()**

Opposite of isnull() — True where a value is NOT NaN.

CODE

```
print(df_na.notnull())
```

OUTPUT

```
Name Age Score
0 True True True
1 True True False
2 True True True
```

■ `df.isnull().sum()`

Counts missing values per column — the go-to first check on any new dataset.

CODE

```
print(df_na.isnull().sum())
```

OUTPUT

```
Name 0
Age 0
Score 1
dtype: int64
```

■ `df.dropna()`

Removes rows (or columns) that contain any NaN value.

CODE

```
print(df_na.dropna())
```

OUTPUT

```
Name Age Score
0 Alice 25 88.5
2 Charlie 35 79.3
```

■ `df.fillna(value)`

Replaces every NaN with a fixed value, a dict of per-column values, or a method.

CODE

```
print(df_na.fillna({'Score': df_na['Score'].mean()}))
```

OUTPUT

```
Name Age Score
0 Alice 25 88.50
1 Bob 30 83.90
2 Charlie 35 79.30
```

■ `mean()` of 88.5 and 79.3 = 83.9 — automatically fills the gap.

■ `df.fillna(method='ffill')`

Forward-fills: copies the last valid value downward to fill NaNs.

CODE

```
print(df_na.fillna(method='ffill'))
```

OUTPUT

```
Name Age Score
0 Alice 25 88.5
1 Bob 30 88.5
2 Charlie 35 79.3
```

■ `df.fillna(method='bfill')`

Backward-fills: copies the next valid value upward to fill NaNs.

CODE

```
print(df_na.fillna(method='bfill'))
```

OUTPUT

```
Name Age Score
0 Alice 25 88.5
1 Bob 30 79.3
2 Charlie 35 79.3
```

■ `df.interpolate()`

Fills NaNs by linear interpolation between surrounding numeric values.

CODE

```
print(df_na.interpolate())
```

OUTPUT

```
Name Age Score
0 Alice 25 88.50
1 Bob 30 83.90
2 Charlie 35 79.30
```

9 · Data Types & Casting

■ **Series.astype(dtype)**

Converts a column to a different data type (int, float, str, bool, etc.).

CODE

```
df['Age'] = df['Age'].astype(float)

print(df.dtypes)
```

OUTPUT

```
Name object
Age float64
Score float64
dtype: object
```

■ **pd.to_numeric()**

Converts a Series to numeric — use errors='coerce' to turn bad values into NaN.

CODE

```
s = pd.Series(['1', '2', 'abc', '4'])

print(pd.to_numeric(s, errors='coerce'))
```

OUTPUT

```
0 1.0
1 2.0
2 NaN
3 4.0
dtype: float64
```

■ **pd.to_datetime()**

Parses strings or numbers into pandas Timestamp (datetime) objects.

CODE

```
s = pd.Series(['2024-01-01', '2024-06-15'])

print(pd.to_datetime(s))
```

OUTPUT

```
0 2024-01-01
1 2024-06-15
dtype: datetime64[ns]
```

■ `pd.Categorical()` / `astype('category')`

Converts a column to the memory-efficient 'category' dtype — ideal for repeated strings.

CODE

```
df['Grade'] = pd.Categorical(['B', 'A', 'C'])

print(df['Grade'].dtype)
```

OUTPUT

```
category
```

■ `Series.unique()`

Returns an array of distinct values (preserves order of first appearance).

CODE

```
print(df['Grade'].unique())
```

OUTPUT

```
['B', 'A', 'C']

Categories (3, object): ['A', 'B', 'C']
```

■ `Series.nunique()`

Returns the count of distinct values — quick cardinality check.

CODE

```
print(df['Grade'].nunique())
```

OUTPUT

```
3
```

■ `Series.value_counts()`

Counts occurrences of each unique value, sorted descending by default.

CODE

```
print(df['Grade'].value_counts())
```

OUTPUT

```
Grade
B  1
A  1
C  1

Name: count, dtype: int64
```

■ **Series.value_counts(normalize=True)**

Same as `value_counts()` but returns relative frequencies (proportions).

CODE

```
print(df['Grade'].value_counts(normalize=True).round(2))
```

OUTPUT

```
Grade
B  0.33
A  0.33
C  0.33

Name: proportion, dtype: float64
```

10 · String Methods (`.str` accessor)

■ **Series.str.lower() / .str.upper()**

Converts all characters to lower-case or upper-case.

CODE

```
print(df['Name'].str.lower())
```

OUTPUT

```
0  alice
1  bob
2  charlie

Name: Name, dtype: object
```

■ **Series.str.strip()**

Removes leading and trailing whitespace from every string.

CODE

```
s = pd.Series([' hello ', ' world '])

print(s.str.strip())
```

OUTPUT

```
0  hello
1  world

dtype: object
```

■ Series.str.replace(old, new)

Replaces occurrences of a substring (or regex) in every string.

CODE

```
print(df['Name'].str.replace('a', '@'))
```

OUTPUT

```
0 Alice # only lowercase a → no match

# use case-insensitive: str.replace('a', '@', case=False)

0 @lice
1 Bob
2 Ch@rlie
```

■ Series.str.contains(pattern)

Returns True if the string contains the given substring (supports regex).

CODE

```
print(df[df['Name'].str.contains('li')])
```

OUTPUT

```
   Name  Age  Score
0  Alice   25   88.5
2  Charlie  35   79.3
```

■ Series.str.startswith() / .str.endswith()

Checks whether each string starts (or ends) with a given prefix/suffix.

CODE

```
print(df['Name'].str.startswith('A'))
```

OUTPUT

```
0 True
1 False
2 False

Name: Name, dtype: bool
```

■ Series.str.len()

Returns the character length of each string.

CODE

```
print(df['Name'].str.len())
```

OUTPUT

```
0 5
1 3
2 7

Name: Name, dtype: int64
```

■ Series.str.split(sep)

Splits each string on a separator — returns a Series of lists.

CODE

```
s = pd.Series(['Alice Smith', 'Bob Jones'])

print(s.str.split(' '))
```

OUTPUT

```
0 [Alice, Smith]
1 [Bob, Jones]

dtype: object
```

■ Series.str.split(expand=True)

Splits and expands the result into separate columns.

CODE

```
s = pd.Series(['Alice Smith', 'Bob Jones'])

print(s.str.split(' ', expand=True))
```

OUTPUT

```
0 1
0 Alice Smith
1 Bob Jones
```

■ **Series.str.get(i)**

Extracts the element at position i from each list/string item.

CODE

```
parts = pd.Series(['Alice Smith', 'Bob Jones']).str.split()

print(parts.str.get(0))
```

OUTPUT

```
0 Alice
1 Bob

dtype: object
```

■ **Series.str.slice(start, stop)**

Slices each string like Python slicing [start:stop].

CODE

```
print(df['Name'].str.slice(0,3))
```

OUTPUT

```
0 Ali
1 Bob
2 Cha

Name: Name, dtype: object
```

■ **Series.str.cat(sep)**

Concatenates all strings in the Series into one string (with a separator).

CODE

```
print(df['Name'].str.cat(sep=', '))
```

OUTPUT

```
Alice, Bob, Charlie
```

11 · DateTime Methods (.dt accessor)

■ `pd.to_datetime()` → `dt.year/month/day`

Extracts year, month, or day components from a datetime Series.

CODE

```
dates = pd.Series(pd.to_datetime(['2024-03-15', '2024-09-01']))

print(dates.dt.year)

print(dates.dt.month)

print(dates.dt.day)
```

OUTPUT

```
0 2024
1 2024
dtype: int32

0 3
1 9
dtype: int32

0 15
1 1
dtype: int32
```

■ `Series.dt.day_name()`

Returns the weekday name ('Monday', 'Tuesday', ...) for each datetime.

CODE

```
print(dates.dt.day_name())
```

OUTPUT

```
0 Friday
1 Sunday
dtype: object
```

■ Series.dt.hour / .dt.minute / .dt.second

Extracts time components from a datetime Series.

CODE

```
ts = pd.Series(pd.to_datetime(['2024-01-01 08:30:45']))

print(ts.dt.hour, ts.dt.minute, ts.dt.second)
```

OUTPUT

```
0 8 dtype: int32

0 30 dtype: int32

0 45 dtype: int32
```

■ Series.dt.date / .dt.time

Extracts the date part or the time part separately from a datetime.

CODE

```
print(ts.dt.date)

print(ts.dt.time)
```

OUTPUT

```
0 2024-01-01

dtype: object

0 08:30:45

dtype: object
```

■ pd.date_range()

Generates a fixed-frequency sequence of dates — great for building time-series indexes.

CODE

```
idx = pd.date_range('2024-01-01', periods=5, freq='D')

print(idx)
```

OUTPUT

```
DatetimeIndex(['2024-01-01', '2024-01-02', '2024-01-03',
               '2024-01-04', '2024-01-05'],
              dtype='datetime64[ns]', freq='D')
```

■ **Series.dt.diff()**

Computes the difference (timedelta) between consecutive datetime values.

CODE

```
print(dates.diff())
```

OUTPUT

```
0 NaT

1 170 days

dtype: timedelta64[ns]
```

12 · Aggregation & Statistics

■ **df.sum()**

Sums each numeric column (axis=0) or each row (axis=1).

CODE

```
print(df[['Age', 'Score']].sum())
```

OUTPUT

```
Age 90.0

Score 259.8

dtype: float64
```

■ **df.mean()**

Computes the arithmetic mean of each numeric column.

CODE

```
print(df[['Age', 'Score']].mean())
```

OUTPUT

```
Age 30.000000

Score 86.600000

dtype: float64
```

■ df.median()

Returns the median (50th percentile) of each numeric column.

CODE

```
print(df[['Age', 'Score']].median())
```

OUTPUT

```
Age  30.0
Score 88.5
dtype: float64
```

■ df.std() / df.var()

Computes standard deviation / variance (ddof=1 by default — sample stats).

CODE

```
print(df['Score'].std().round(2))

print(df['Score'].var().round(2))
```

OUTPUT

```
6.38

40.72
```

■ df.min() / df.max()

Returns the minimum / maximum value of each column.

CODE

```
print(df[['Age', 'Score']].min())

print(df[['Age', 'Score']].max())
```

OUTPUT

```
Age  25.0
Score 79.3
dtype: float64

Age  35.0
Score 92.0
dtype: float64
```

■ df.count()

Counts non-NaN values per column — differ from len(df) when NaNs exist.

CODE

```
print(df.count())
```

OUTPUT

```
Name 3
Age 3
Score 3
dtype: int64
```

■ df.cumsum() / df.cumprod()

Computes a running cumulative sum / product along a column.

CODE

```
print(df['Score'].cumsum())
```

OUTPUT

```
0 88.5
1 180.5
2 259.8
Name: Score, dtype: float64
```

■ df.pct_change()

Computes the percentage change between consecutive rows — common in finance.

CODE

```
print(df['Score'].pct_change().round(3))
```

OUTPUT

```
0 NaN
1 0.040
2 -0.138
Name: Score, dtype: float64
```

■ df.quantile(q)

Returns the value at the q-th quantile (0–1) for each numeric column.

CODE

```
print(df['Score'].quantile([0.25, 0.75]))
```

OUTPUT

```
0.25 83.9
0.75 90.25
Name: Score, dtype: float64
```

■ df.corr()

Computes the pairwise Pearson correlation matrix for numeric columns.

CODE

```
print(df[['Age', 'Score']].corr().round(2))
```

OUTPUT

```
Age Score
Age 1.00 0.99
Score 0.99 1.00
```

■ df.agg()

Applies one or more aggregation functions in a single call.

CODE

```
print(df[['Age', 'Score']].agg(['min', 'max', 'mean']).round(1))
```

OUTPUT

```
Age Score
min 25.0 79.3
max 35.0 92.0
mean 30.0 86.6
```

■ df.nunique()

Counts distinct values per column — useful to spot low-cardinality columns.

CODE

```
print(df.nunique())
```

OUTPUT

```
Name 3
Age 3
Score 3
dtype: int64
```

■ df.abs()

Returns element-wise absolute values for numeric columns.

CODE

```
s = pd.Series([-5, 3, -8, 2])

print(s.abs())
```

OUTPUT

```
0 5
1 3
2 8
3 2
dtype: int64
```

■ df.round(decimals)

Rounds each numeric value to the given number of decimal places.

CODE

```
print(df['Score'].round(0))
```

OUTPUT

```
0 88.0
1 92.0
2 79.0

Name: Score, dtype: float64
```

■ **df.clip(lower, upper)**

Clips values below lower to lower, and above upper to upper.

CODE

```
print(df['Score'].clip(80, 90))
```

OUTPUT

```
0  88.5
1  90.0
2  80.0

Name: Score, dtype: float64
```

13 · Applying Functions (apply / map)

■ **Series.map(dict or function)**

Maps each value through a dict or function — perfect for encoding labels.

CODE

```
grade_map = {'A':4, 'B':3, 'C':2}

df['GPA'] = df['Grade'].map(grade_map)

print(df[['Grade', 'GPA']])
```

OUTPUT

```
Grade GPA
0  B    3
1  A    4
2  C    2
```

■ df.apply(func, axis)

Applies a function along rows (axis=1) or columns (axis=0) of a DataFrame.

CODE

```
def classify(row):  
  
    return 'Senior' if row['Age'] >= 30 else 'Junior'  
  
df['Level'] = df.apply(classify, axis=1)  
  
print(df[['Name', 'Level']])
```

OUTPUT

```
   Name  Level  
0  Alice  Junior  
1   Bob  Senior  
2  Charlie  Senior
```

■ Series.apply(func)

Applies a function element-by-element to a Series.

CODE

```
df['Score_sq'] = df['Score'].apply(lambda x: round(x**2, 1))  
  
print(df[['Score', 'Score_sq']].head(3))
```

OUTPUT

```
   Score  Score_sq  
0  88.5   7832.2  
1  92.0   8464.0  
2  79.3   6288.5
```

■ `df.applymap()` → `df.map()` in pandas 2.1+

Applies a function to every individual cell in a DataFrame.

CODE

```
# pandas 2.1+

result = df[['Age', 'Score']].map(lambda x: round(x,0))

print(result)
```

OUTPUT

```
Age Score
0  25.0  88.0
1  30.0  92.0
2  35.0  79.0
```

14 · Merging & Joining

■ `pd.merge(df1, df2, on=key)`

SQL-style join on a common key column (inner join by default).

CODE

```
orders = pd.DataFrame({'ID':[1,2,3], 'Item':['Pen', 'Book', 'Bag']})

details = pd.DataFrame({'ID':[1,3], 'Price':[10,250]})

print(pd.merge(orders, details, on='ID'))
```

OUTPUT

```
ID Item Price
0  1  Pen   10
1  3  Bag  250
```

■ `pd.merge(..., how='left')`

Left join — keeps all rows from the left DataFrame; NaN for missing right values.

CODE

```
print(pd.merge(orders, details, on='ID', how='left'))
```

OUTPUT

```
ID Item Price
0 1 Pen 10.0
1 2 Book NaN
2 3 Bag 250.0
```

■ `pd.merge(..., how='outer')`

Full outer join — keeps all rows from both DataFrames; NaN where no match.

CODE

```
print(pd.merge(orders, details, on='ID', how='outer'))
```

OUTPUT

```
ID Item Price
0 1 Pen 10.0
1 2 Book NaN
2 3 Bag 250.0
```

■ `df.join(other, on=key)`

Joins two DataFrames on their index (or a key column) — simpler API than merge.

CODE

```
df1 = pd.DataFrame({'Score':[88,92]}, index=['Alice','Bob'])
df2 = pd.DataFrame({'City':['Delhi','Mumbai']}, index=['Alice','Bob'])
print(df1.join(df2))
```

OUTPUT

```
Score City
Alice 88 Delhi
Bob 92 Mumbai
```

■ `pd.merge_asof()`

Merges on a sorted key by nearest match — useful for time-series look-ups.

CODE

```
trades = pd.DataFrame({'time':[1,5,10], 'price':[100,105,110]})

quotes = pd.DataFrame({'time':[2,7], 'bid': [ 99,104]})

print(pd.merge_asof(trades, quotes, on='time'))
```

OUTPUT

```
time price bid
0 1 100 NaN
1 5 105 99.0
2 10 110 104.0
```

15 · Reshaping (pivot / melt / stack / unstack)

■ `df.pivot(index, columns, values)`

Reshapes long-format data into a wide table — each unique column value becomes a column.

CODE

```
long = pd.DataFrame({

    'Student':['Alice','Alice','Bob','Bob'],

    'Subject':['Math','Sci','Math','Sci'],

    'Score': [90, 85, 78, 92]

})

wide = long.pivot(index='Student', columns='Subject', values='Score')

print(wide)
```

OUTPUT

```
Subject Math Sci
Student
Alice 90 85
Bob 78 92
```

■ **df.pivot_table(values, index, columns, aggfunc)**

Like pivot but handles duplicate entries by aggregating (sum, mean, etc.).

CODE

```
pt = long.pivot_table(values='Score', index='Student',  
  
columns='Subject', aggfunc='mean')  
  
print(pt)
```

OUTPUT

```
Subject Math Sci  
  
Student  
  
Alice 90 85  
  
Bob 78 92
```

■ **pd.melt(df, id_vars, value_vars)**

Opposite of pivot — turns wide-format columns into long-format rows.

CODE

```
print(pd.melt(wide.reset_index(),  
  
id_vars='Student',  
  
value_vars=['Math', 'Sci']))
```

OUTPUT

```
Student Subject value  
  
0 Alice Math 90  
  
1 Bob Math 78  
  
2 Alice Sci 85  
  
3 Bob Sci 92
```

■ df.stack()

Moves the innermost column level into the row index — makes data longer.

CODE

```
stacked = wide.stack()

print(stacked.head(4))
```

OUTPUT

```
Student Subject
Alice Math 90
      Sci 85
Bob Math 78
     Sci 92
dtype: int64
```

■ df.unstack()

Moves the innermost row-index level back out into columns — reverses stack().

CODE

```
print(stacked.unstack())
```

OUTPUT

```
Subject Math Sci
Student
Alice 90 85
Bob 78 92
```

■ df.transpose() / df.T

Flips the DataFrame so rows become columns and columns become rows.

CODE

```
print(df[['Age', 'Score']].head(2).T)
```

OUTPUT

```
0 1
Age 25.0 30.0
Score 88.5 92.0
```

■ df.explode(col)

Expands a column that contains lists — each list item gets its own row.

CODE

```
df_ex = pd.DataFrame({'Name': ['Alice', 'Bob'], 'Tags': [['a', 'b'], ['c']]})

print(df_ex.explode('Tags'))
```

OUTPUT

```
   Name Tags
0  Alice  a
0  Alice  b
1   Bob   c
```

■ df.wide_to_long()

Reshapes a DataFrame from wide format to long format using a stubname pattern.

CODE

```
wide2 = pd.DataFrame({'id': [1, 2], 'val_2020': [10, 20], 'val_2021': [11, 21]})

print(pd.wide_to_long(wide2, stubnames='val', i='id', j='year', sep='_'))
```

OUTPUT

```
   val
id year
1 2020 10
2 2020 20
1 2021 11
2 2021 21
```

■ df.crosstab()

Builds a frequency (or aggregated) cross-tabulation table of two or more factors.

CODE

```
ct = pd.crosstab(df['Grade'], df['Level'])

print(ct)
```

OUTPUT

```
Level Junior Senior
Grade
A    0    1
B    1    0
C    0    1
```

16 · Binning with `pd.cut()`

■ `pd.cut(series, bins)`

Divides a continuous numeric column into discrete intervals (bins).

CODE

```
ages = pd.Series([15, 22, 35, 48, 60, 72])

bins = [0, 18, 35, 60, 100]

labels= ['Teen', 'Young', 'Mid', 'Senior']

result = pd.cut(ages, bins=bins, labels=labels)

print(result)
```

OUTPUT

```
0 Teen
1 Young
2 Young
3 Mid
4 Mid
5 Senior

dtype: category
```

■ Intervals are *(left, right]* by default — right endpoint is included.

■ `pd.cut()` with `right=False`

Makes intervals [left, right) — left endpoint is included instead.

CODE

```
r = pd.cut(ages, bins=bins, labels=labels, right=False)

print(r)
```

OUTPUT

```
0 Teen
1 Young
2 Mid
3 Mid
4 Senior
5 Senior
dtype: category
```

■ `pd.cut()` — `value_counts()` on result

Combine with `value_counts()` to instantly see how many fall in each bin.

CODE

```
print(pd.cut(ages, bins=bins, labels=labels).value_counts())
```

OUTPUT

```
Young 2
Mid 2
Teen 1
Senior 1
dtype: int64
```

■ `pd.cut()` — `retbins=True`

Also returns the computed bin edges — useful when `bins` is an integer count.

CODE

```
result, edges = pd.cut(ages, bins=3, retbins=True)

print(edges.round(1))
```

OUTPUT

```
[ 14.9 33.7 52.3 72. ]
```

■ With `bins=3`, pandas creates 3 equal-width intervals automatically.

■ **pd.qcut(series, q)**

Like `pd.cut()` but creates quantile-based bins so each bin has equal number of items.

CODE

```
scores = pd.Series([55, 68, 72, 80, 85, 90, 95, 99])

quartiles = pd.qcut(scores, q=4, labels=['Q1', 'Q2', 'Q3', 'Q4'])

print(quartiles)
```

OUTPUT

```
0  Q1
1  Q1
2  Q2
3  Q2
4  Q3
5  Q3
6  Q4
7  Q4

dtype: category
```

■ *qcut guarantees roughly equal group sizes; cut guarantees equal bin widths.*

17 · GroupBy

■ df.groupby(col).agg(func)

Splits data by a key column, applies an aggregation, and combines the results.

CODE

```
data = pd.DataFrame({  
    'Dept': ['HR', 'IT', 'HR', 'IT', 'HR'],  
    'Name': ['Ann', 'Ben', 'Cara', 'Dan', 'Eve'],  
    'Salary': [40, 70, 45, 80, 42]  
})  
  
print(data.groupby('Dept')['Salary'].mean())
```

OUTPUT

```
Dept  
HR 42.333333  
IT 75.000000  
Name: Salary, dtype: float64
```

■ df.groupby().sum() / .count() / .max()

All standard aggregation methods work directly on a GroupBy object.

CODE

```
print(data.groupby('Dept')['Salary'].sum())  
  
print(data.groupby('Dept')['Salary'].max())
```

OUTPUT

```
Dept  
HR 127  
IT 150  
Name: Salary, dtype: int64  
  
Dept  
HR 45  
IT 80  
Name: Salary, dtype: int64
```

■ **df.groupby().agg({'col': ['min','max','mean']})**

Apply different aggregations to different columns in one call.

CODE

```
result = data.groupby('Dept').agg(  
  
    Min_Sal=('Salary', 'min'),  
  
    Max_Sal=('Salary', 'max'),  
  
    Headcount=('Name', 'count')  
  
)  
  
print(result)
```

OUTPUT

```
Min_Sal  Max_Sal  Headcount  
Dept  
HR      40      45      3  
IT      70      80      2
```

■ **df.groupby().size()**

Returns the number of rows in each group (including NaN rows).

CODE

```
print(data.groupby('Dept').size())
```

OUTPUT

```
Dept  
HR      3  
IT      2  
  
dtype: int64
```

■ `df.groupby().apply(func)`

Applies a custom function to each group — most flexible GroupBy method.

CODE

```
def top_earner(grp):  
  
    return grp.loc[grp['Salary'].idxmax(), 'Name']  
  
print(data.groupby('Dept').apply(top_earner))
```

OUTPUT

```
Dept  
HR Cara  
IT Dan  
dtype: object
```

■ `df.groupby().transform(func)`

Applies a function per group but returns a result with the same shape as the input.

CODE

```
data['Dept_mean'] = data.groupby('Dept')['Salary'].transform('mean')  
  
print(data[['Name', 'Dept', 'Salary', 'Dept_mean']])
```

OUTPUT

```
Name Dept Salary Dept_mean  
0 Ann HR 40 42.333333  
1 Ben IT 70 75.000000  
2 Cara HR 45 42.333333  
3 Dan IT 80 75.000000  
4 Eve HR 42 42.333333
```

■ *transform is perfect for normalisation or adding group-level stats as a new column.*

■ df.groupby().filter(func)

Keeps only the groups where the function returns True.

CODE

```
big_depts = data.groupby('Dept').filter(lambda g: len(g) >= 3)

print(big_depts)
```

OUTPUT

```
Dept Name Salary
0  HR  Ann  40
2  HR  Cara  45
4  HR  Eve  42
```

■ df.groupby(multiple cols)

Group by two or more columns simultaneously — creates a MultiIndex result.

CODE

```
data['Level'] = ['Jr', 'Sr', 'Sr', 'Sr', 'Jr']

result = data.groupby(['Dept', 'Level'])['Salary'].mean()

print(result)
```

OUTPUT

```
Dept Level
HR  Jr  41.0
    Sr  45.0
IT  Sr  75.0

Name: Salary, dtype: float64
```

■ GroupBy with pd.cut() — binned grouping

Combine pd.cut() with groupby to aggregate over numeric ranges.

CODE

```
scores_df = pd.DataFrame({
    'Student': ['A', 'B', 'C', 'D', 'E'],
    'Score': [45, 67, 82, 91, 55]
})

bins = [0, 60, 80, 100]

labels = ['Fail', 'Pass', 'Distinction']

scores_df['Grade'] = pd.cut(scores_df['Score'],
    bins=bins, labels=labels)

print(scores_df.groupby('Grade', observed=True)['Score'].agg(['count', 'mean']))
```

OUTPUT

```
count mean
Grade
Fail 2 50.00000
Pass 1 67.00000
Distinction 2 86.50000
```

■ *observed=True suppresses empty category groups in pandas 2.x.*

■ You've covered every core Pandas function!

From `pd.DataFrame()` all the way through `groupby` and `pd.cut()` — no function skipped.
Keep this as your daily reference cheatsheet.

Quick Cheat Sheet	
Create	<code>pd.DataFrame()</code> , <code>pd.Series()</code> , <code>pd.read_csv()</code>
View	<code>df.head()</code> , <code>df.info()</code> , <code>df.describe()</code>
Select	<code>df['col']</code> , <code>df.loc[]</code> , <code>df.iloc[]</code>
Filter	<code>df[mask]</code> , <code>df.query()</code> , <code>.isin()</code> , <code>.between()</code>
Clean	<code>df.dropna()</code> , <code>df.fillna()</code> , <code>df.interpolate()</code>
Transform	<code>df.apply()</code> , <code>Series.map()</code> , <code>df.assign()</code>
Merge	<code>pd.merge()</code> , <code>pd.concat()</code> , <code>df.join()</code>
Reshape	<code>df.pivot()</code> , <code>pd.melt()</code> , <code>df.stack()</code>
Aggregate	<code>df.groupby().agg()</code> , <code>df.sum()</code> , <code>df.corr()</code>
Bin	<code>pd.cut()</code> , <code>pd.qcut()</code>